NED UNIVERSITY
OF ENGINEERING AND TECHNOLOGY
KARACHI, SINDH, PAKISTAN

DEPARTMENT OF CSIT

Course Name:

Programming For AI

Course Code:

CT-264

Semester: Fall 2026 | **Batch:** 2024

Section: "E (Cyber Security)"

COMPLEX COMPUTING ACTIVITY (CCA)

AI-BASED PATIENT ADMISSION PREDICTION SYSTEM

SUBMITTED BY:

MUHAMMAD HAMZA SIDDIQUI (CT-24033)
SAIF UR REHMAN (CT-24044)
MUHAMMAD TALHA BIN SUHAIL (CT-24312)

SUBMITTED TO:

Miss Amna Ahmed

TABLE OF CONTENTS

Abstract 4

1. Introduction 4

1.1 Background 4

1.2 Problem Statement 4

1.3 Objectives 4

1.4 Real-World Significance 5

1.5 Scope 5

2. Problem Analysis 6

2.1 Inputs and Outputs 6

2.2 Alternative Solution Strategies 6

2.3 Success Criteria 7

3. Datasets 7

3.1 Dataset 1 - Emergency Department Patient Flow (USA) 7

3.2 Dataset 2 - Korean Triage and Acuity Scale (KTAS) 8

3.3 Relevance to the Problem 9

4. Methodology 9

5. Data Preprocessing 11

5.1 Data Quality Problems Found 11

5.2 Handling Missing Values 12

5.3 Encoding, Scaling and Pipelines 12

6. Exploratory Data Analysis 13

7. Class Imbalance 19

8. Models Used 20

8.1 Logistic Regression 20

8.2 Decision Tree 20

8.3 Random Forest 20

8.4 Gradient Boosting (HistGradientBoostingClassifier) 20

8.5 Hyperparameter Tuning 20

8.6 Ensemble (Soft Voting) 21

9. Results and Evaluation 21

9.1 Model Comparison 21
9.2 ROC and Precision-Recall Curves 22
9.3 Confusion Matrices 23
9.4 Decision-Threshold Tuning 23
9.5 Feature Importance 24
9.6 Data-Leakage Check 24
10. Dataset 2 and Cross-Dataset Validation 25
10.1 Korean KTAS Dataset 25
10.2 External Validation (USA → Korea) 26
11. Visualization and User Interface 26
11.1 Admission Prediction App 27
11.2 Bed-Demand Forecast 28
12. Challenges and Solutions 29
13. Ethical Considerations and Limitations 29
14. Conclusion and Future Work 30
References 30
Appendix A: Project Files and How to Run 31
Appendix B: Key Code 31

Abstract

Emergency departments (EDs) are crowded because the decision to admit a patient is usually made hours after arrival, and only then does the search for a hospital bed begin. This project builds an **AI-Based Patient Admission Prediction System** that predicts, at the moment of triage, whether a patient will be **admitted** or **discharged**. Two public, de-identified datasets are used: an ED patient-flow dataset from the Yale New Haven Health System (100,000 visits sampled from 560,486) and the Korean Triage and Acuity Scale (KTAS) dataset (1,267 visits). The data was cleaned (placeholder answers, inconsistent spelling, impossible vital signs, text inside numeric columns, missing values) and explored with statistical summaries and 19 charts. The class imbalance (29.8% admitted) was handled with stratified splitting, balanced class weights and threshold tuning. Four classifiers (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting) were trained and improved by hyperparameter tuning and a soft-voting ensemble. The final **Tuned Gradient Boosting** model reached **ROC-AUC 0.885**, recall 0.829, F1-score 0.703 and accuracy 0.792 on 20,000 unseen visits. A data-leakage problem (home-medication lists completed during the admission) was detected and removed. The model was also validated on the Korean dataset (ROC-AUC 0.760) and a desktop interface was built for ED staff that shows the risk level, a recommended action and a calibrated bed-demand forecast.

1. Introduction

1.1 Background

Emergency departments work around the clock and receive patients with very different needs: some only need a prescription, while others need intensive care. When an ED becomes crowded, waiting times grow, ambulances may be diverted and patients who need a ward bed wait for hours on ED trolleys ("boarding"). Studies have linked crowding and boarding with delayed treatment and worse outcomes. A major cause is that bed management only starts after the doctor has finished the assessment, which can take several hours after arrival.

At triage, a nurse already records the patient's age, arrival mode, chief complaint, vital signs and an urgency level (ESI in the USA, KTAS in Korea). Hospital records also contain the patient's history, such as previous visits and known diseases. If a machine-learning model can use this early information to estimate who is likely to be admitted, the hospital can request beds, call specialists and plan staff much earlier.

1.2 Problem Statement

Given the information that is available **at triage** (demographics, arrival details, triage level, vital signs, chief complaint and hospital history), predict whether the ED visit will end in **hospital admission (1)** or **discharge (0)**. This is a supervised **binary classification** problem on tabular data with a moderately imbalanced target.

1.3 Objectives

- Collect and describe suitable real-world datasets for ED admission prediction (two datasets).

- Clean the data: handle missing values, placeholders, inconsistent categories and impossible values.
- Perform exploratory data analysis (EDA) to find the factors linked to admission.
- Check and handle class imbalance.
- Train and compare several classification models and improve them with tuning and ensembling.
- Evaluate the models with suitable metrics (precision, recall, F1, ROC-AUC, PR-AUC, confusion matrix).
- Test how well the approach generalises to a second hospital system in another country.
- Provide a simple, user-friendly interface that turns predictions into actions for hospital staff.

1.4 Real-World Significance

Early admission prediction supports **hospital operations**: bed managers can reserve ward beds, nurses can start the admission paperwork, and managers can schedule staff based on the expected number of admissions per shift. Low-risk patients can be moved to a fast-track area, which shortens waiting time for everyone. The same idea is relevant to busy public hospitals in Pakistan, where emergency rooms are often over capacity and beds are limited.

1.5 Scope

The system is a decision-support prototype built within the scope of an introductory Programming for AI course. It uses classical machine-learning models in Python (scikit-learn) and a Tkinter desktop interface. It does not replace clinical judgement; real deployment would need local validation, ethics approval and integration with the hospital information system.

2. Problem Analysis

2.1 Inputs and Outputs

Table 1: *Inputs and outputs of the system*

Group	Inputs used by the model	Data type
Patient	Age, gender, race, ethnicity, language, marital & employment status, insurance	Numeric / categorical
Arrival	ED site, arrival mode (car, ambulance, walk-in...), month, weekday, time band	Categorical
Triage	ESI level 1-5, chief complaint (top 30 + other)	Ordinal / categorical
Vital signs	Heart rate, systolic & diastolic BP, respiratory rate, SpO2, temperature, oxygen device	Numeric (with missing values)
History	ED visits, admissions, surgeries, imaging tests and ECGs in previous records, previous ED outcome	Numeric / categorical
Comorbidities	Number of past conditions + 10 flags (hypertension, diabetes, heart failure, COPD...)	Numeric / binary
OUTPUT	**Admission score (0-1), predicted class (Admit / Discharge), calibrated probability, risk band (LOW / MODERATE / HIGH) and a recommended action**	**Probability + class**

2.2 Alternative Solution Strategies

Several strategies were compared before choosing the final approach:

Table 2: *Alternative strategies that were evaluated*

Strategy	Advantages	Disadvantages	Decision
Triage level only (rule)	Very simple; already used by nurses	Ignores age, history and vitals; ROC-AUC only 0.71 on the KTAS data	Used as baseline
Logistic Regression	Fast, interpretable coefficients	Assumes linear effects	Implemented
Decision Tree	Easy to explain as rules	Unstable, over-fits	Implemented
Random Forest	Strong on tabular data, robust	Slower, larger model	Implemented
Gradient Boosting	Usually the best accuracy on tabular data, handles interactions	Needs tuning	Implemented + tuned (final)
Ensemble (voting)	Combines strengths of models	More complex	Implemented
Deep neural network	Flexible	Needs more data/tuning, less interpretable, small gain on tabular data	Not chosen (scope)

2.3 Success Criteria

- ROC-AUC above 0.85 on unseen test data (good discrimination).
- Recall of at least 0.80 for admitted patients, because missing an admission (false negative) is more harmful for bed planning than a false alarm.
- Only information available at triage is used (no data leakage).
- The full pipeline runs on a normal laptop in a few minutes.

3. Datasets

The project brief requires at least two relevant datasets or one sufficiently large dataset. This project uses **both**: one large dataset for model development and a second, independent dataset from another country for validation.

3.1 Dataset 1 - Emergency Department Patient Flow (USA)

Source: Hong, Haimovich and Taylor (2018), "Predicting hospital admission at emergency department triage using machine learning", PLOS ONE 13(7). The de-identified data was released publicly on GitHub/Zenodo (yaleemmlc/admissionprediction) and is also shared on Kaggle. It covers adult ED visits to three emergency departments of the Yale New Haven Health System (2014-2017).

The original file contains **560,486 visits and 972 columns** (one column per chief complaint, medication class, lab test, etc.). The script 01_prepare_datasets.py turned it into a manageable analysis file: the 200 chief-complaint columns were merged into one categorical column, 281 diagnosis columns were summarised into a comorbidity count and ten clinically important flags, and a random sample of **100,000 visits** was taken (random seed 42) so the project runs on a laptop. The admission rate of the sample (29.8%) matches the full dataset (29.7%).

Table 3: *Structure of Dataset 1 (ed_patient_flow.csv, 100,000 rows x 42 columns)*

Group	Columns	Type	Meaning
Identifier	visit_id	Integer	Row number (not used for learning)
Demographics	age, gender, race, ethnicity, language, marital_status, employment_status, insurance_status	Numeric / text	Who the patient is
Arrival	ed_site, arrival_mode, arrival_month, arrival_day, arrival_hour_bin	Text	How and when the patient came
Triage	esi_level, chief_complaint	Ordinal / text	Urgency and main problem
Vital signs	heart_rate, systolic_bp, diastolic_bp, respiratory_rate, oxygen_saturation, on_supplemental_oxygen, temperature_f	Float	Physiological condition
History	previous_disposition, prior_ed_visits, prior_admissions, prior_surgeries, prior_imaging_tests, prior_ekgs, home_medications	Numeric / text	Past use of the hospital
Comorbidities	num_comorbidities, hypertension, diabetes, heart_failure, coronary_disease, arrhythmia, copd_asthma, kidney_disease, cancer_history, mental_health_disorder, substance_use_disorder	Integer / binary	Chronic illness burden
Target	disposition (Admit / Discharge)	Text	What we predict

3.2 Dataset 2 - Korean Triage and Acuity Scale (KTAS)

Source: Moon, Shim, Park and Park (2019), "Triage accuracy and causes of mistriage using the Korean Triage and Acuity Scale", PLOS ONE 14(9) - supporting data file S1 (also on Kaggle as "Emergency Service - Triage Application"). It contains **1,267 adult ED visits** from two emergency departments in Korea (October 2016 - September 2017) with **24 columns**.

Table 4: *Structure of Dataset 2 (ktas_triage.csv)*

Group	Columns	Type
Patient	Sex, Age, Group (local / regional ED)	Coded integers / float
Arrival	Arrival mode (walking, 119 ambulance, private car...), Patients number per hour	Coded integers
Clinical	Injury, Mental state, Pain, NRS_pain, SBP, DBP, HR, RR, BT, Saturation	Mixed (numbers stored as text)
Triage	KTAS_RN (nurse), KTAS_expert, Error_group, mistriage, KTAS duration	Integers
Free text	Chief_complain, Diagnosis in ED	Text
Outcome	Disposition: 1 discharge, 2 ward, 3 ICU, 4 AMA discharge, 5 transfer, 6 death, 7 operation	Coded integers

The target was defined as **admitted = 1** for ward admission, ICU admission or operation from ED, and **0** for discharge or discharge against medical advice. Transfers to another hospital and deaths (41 visits) were removed because they are neither an admission here nor a discharge. This leaves 1,226 visits, of which 403 (32.9%) were admitted. Columns that are only known after the visit (Diagnosis, Length of stay, KTAS_expert, mistriage) were not used as features to avoid leakage.

3.3 Relevance to the Problem

Both datasets record what is known when the patient arrives and what finally happened (admission or discharge), which is exactly the input-output relationship the system must learn. Dataset 1 is large and rich, which is ideal for training and comparing models. Dataset 2 comes from a different country, triage scale and hospital system, so it shows whether the approach is general and reveals real data-quality problems (such as text inside numeric columns).

4. Methodology

Figure 1 shows the complete workflow. Each block corresponds to a section of the main script 02_admission_prediction.py, and every step is implemented with pandas and scikit-learn Pipelines so that the same transformations are applied during training, testing and in the user interface.

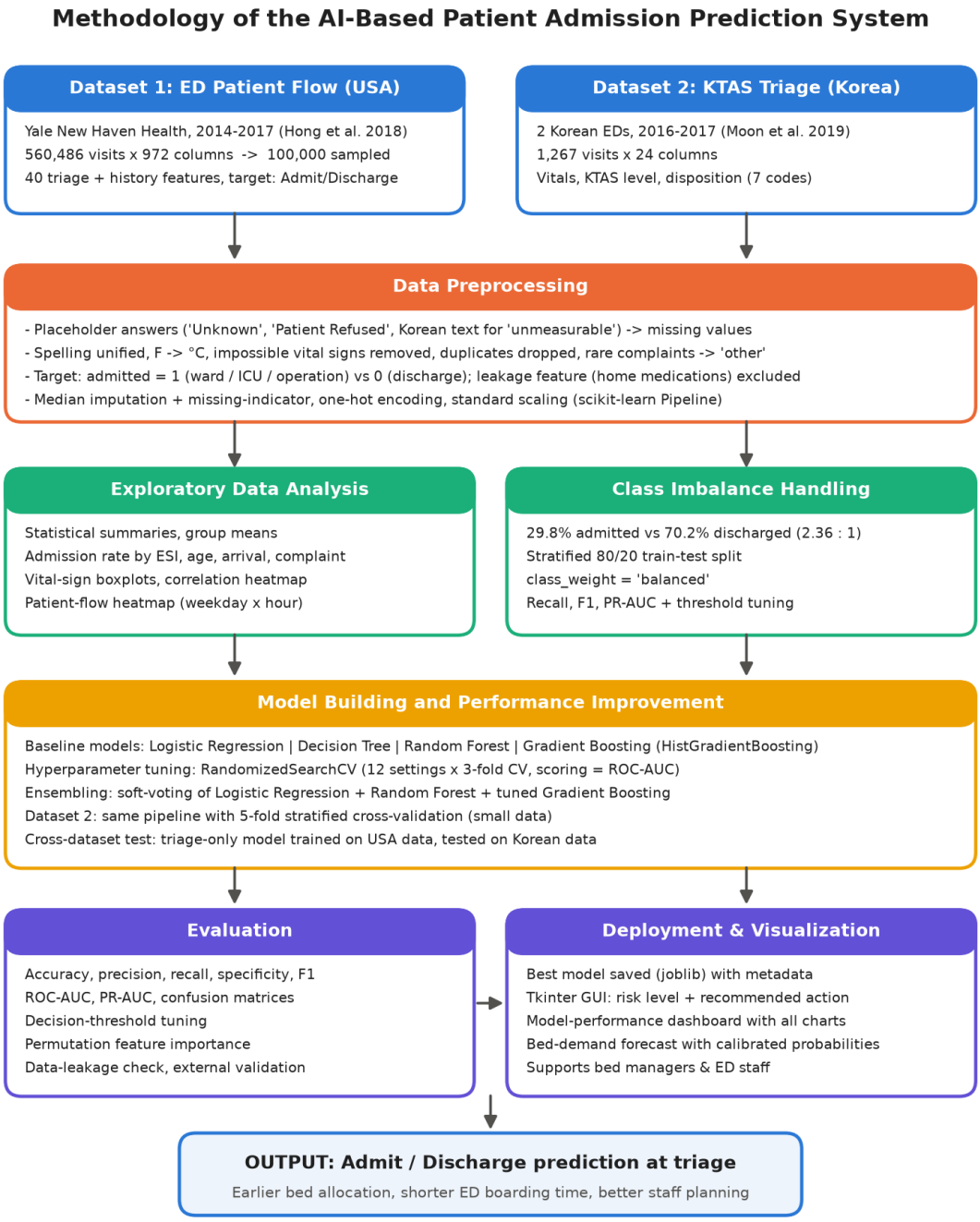


Figure 1: *Methodology of the AI-Based Patient Admission Prediction System*

Table 5: *Summary of the methodology steps*

Step	Description
1. Data acquisition	Download both datasets; 01_prepare_datasets.py converts them to CSV files.
2. Data understanding	Shape, types, duplicates, missing values, placeholders, value ranges.
3. Preprocessing	Clean categories and numbers, convert units, build target, impute, encode, scale.
4. EDA	Statistics and 11 charts showing how each factor relates to admission.
5. Imbalance handling	Stratified split, balanced class weights, minority-focused metrics, threshold tuning.
6. Modelling	Logistic Regression, Decision Tree, Random Forest, Gradient Boosting (80% training data).
7. Improvement	RandomizedSearchCV tuning + soft-voting ensemble; data-leakage check.
8. Evaluation	Test set of 20,000 unseen visits; ROC/PR curves, confusion matrices, feature importance.
9. Validation	Dataset 2 cross-validation and USA-to-Korea external validation.
10. Deployment	Model saved with joblib; Tkinter GUI and bed-demand forecast for hospital staff.

Tools and libraries: Python 3.14.6, pandas 3.0.3, NumPy 2.4.6, scikit-learn 1.9.1, matplotlib 3.11.0, seaborn 0.13.2, joblib (model saving) and Tkinter (graphical interface).

5. Data Preprocessing

5.1 Data Quality Problems Found

A data-quality report was produced before any modelling. Figure 2 shows the percentage of missing values per column in Dataset 1. Vital signs are missing for about 30% of visits and oxygen saturation for almost half, because not every vital sign is recorded for every patient at triage.

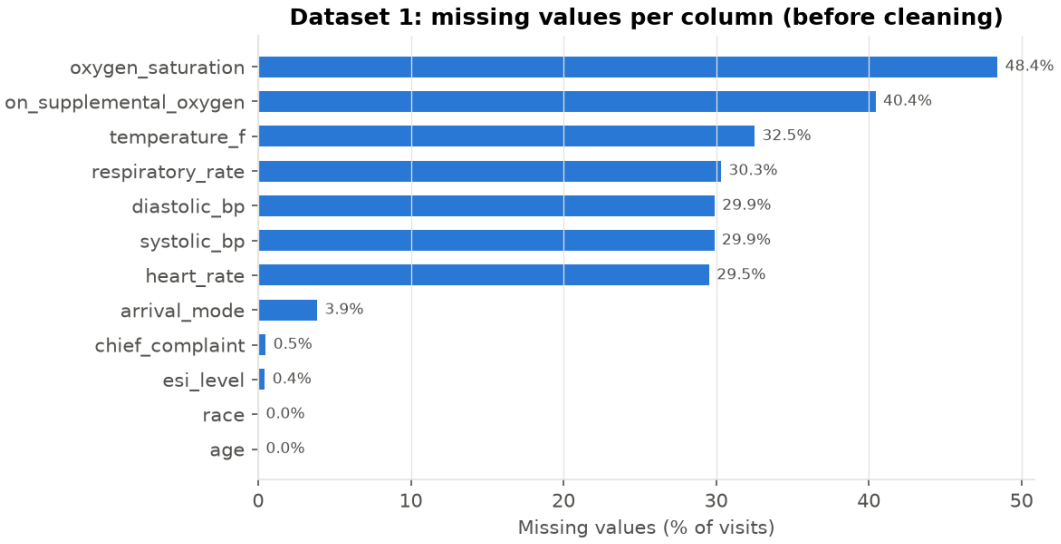


Figure 2: *Missing values per column in Dataset 1 (before cleaning)*

Table 6: *Data-quality problems and how they were handled*

Problem	Example / amount	Solution
Missing vital signs	heart_rate 29.5%, oxygen_saturation 48.4% missing	Median imputation + 'was missing' indicator column
Missing triage / arrival data	esi_level 0.4%, arrival_mode 3.9% missing	Median (numbers) or the category 'Missing' (text)
Placeholder answers	'Unknown' and 'Patient Refused' in race, ethnicity, marital and employment status	Converted to missing values
Inconsistent spelling	'ambulance' (lower case) next to 'Car', 'Walk-in'	Unified to title case ('Ambulance', 'Walk-In')
Different units	Temperature in °F (USA) vs °C (Korea)	Converted to °C
Impossible values	Heart rate 280 bpm, respiratory rate 69/min, DBP 214 mmHg	Values outside physiological limits set to missing (8 values)
Duplicates	Identical visit records	1 duplicate removed
Too many categories	200 different chief complaints	Top 30 kept, rest grouped as 'other'
Text in numeric columns (Dataset 2)	Korean word '측불' (= unmeasurable) in vital signs, e.g. SBP 25, DBP 29, HR 20 rows	Converted to numbers; text becomes missing
Structural missing (Dataset 2)	NRS pain score empty when patient has no pain	Filled with 0 when Pain = 'no pain'
Data leakage	home_medications: 6.1 for admitted vs 0.5 for discharged	Excluded from the model (see Section 9.6)

5.2 Handling Missing Values

Rows with missing values were **not deleted**, because that would remove about half of the data and bias the sample. Numeric columns were filled with the **median** (robust to outliers), and for every column with missing values an extra **missing-indicator** column was added. This is important because missingness itself carries information: patients without a recorded heart rate were admitted in 43.6% of cases, compared with 24.0% when it was recorded, and for oxygen saturation the figures are 34.7% vs 25.2%. Missing text values received their own category 'Missing'.

5.3 Encoding, Scaling and Pipelines

Text columns were converted with **one-hot encoding** (unknown categories in new data are ignored), and numeric columns were **standardised** (mean 0, standard deviation 1), which Logistic Regression needs. All steps were placed inside a scikit-learn **ColumnTransformer + Pipeline**. The pipeline is fitted on the training data only, so no information from the test set leaks into the medians or scaling, and the saved model applies exactly the same preprocessing to new patients in the GUI.

Code 1: preprocessing pipeline (02_admission_prediction.py)

```
def make_preprocessor(numeric_features=NUMERIC_FEATURES):
    numeric = Pipeline([
        ("impute", SimpleImputer(strategy="median", add_indicator=True)),
        ("scale", StandardScaler()),
    ])
    categorical = Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="Missing")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ])
```

```
    return ColumnTransformer([
        ("num", numeric, numeric_features),
        ("cat", categorical, CATEGORICAL_FEATURES),
    ])
```

6. Exploratory Data Analysis

EDA was carried out on the cleaned Dataset 1 (99,999 visits). Table 7 compares the average values of important features for admitted and discharged patients.

Table 7: *Mean feature values by outcome*

Feature	Admitted (mean)	Discharged (mean)
age	61.64	44.85
esi_level	2.43	3.27
prior_admissions	1.62	0.64
num_comorbidities	8.15	4.71
heart_rate	88.89	84.54
oxygen_saturation	96.62	97.47
temperature_c	36.76	36.70

Admitted patients are on average **17 years older**, have a more urgent triage level (lower ESI), more previous admissions, nearly twice as many past conditions, a faster heart rate and lower oxygen saturation.

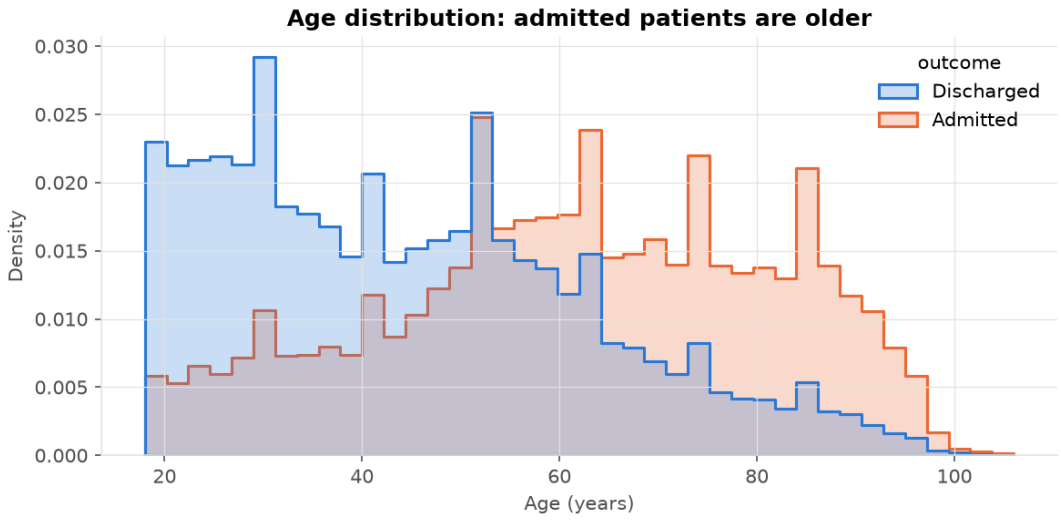


Figure 3: *Age distribution of admitted and discharged patients*

Figure 3 shows that most discharged patients are young adults, while the admitted group peaks after 60 years. The admission rate rises steadily with age: 11.0% for 18-30 year olds, 30.1% for 46-60 and 61.6% for patients aged 76 or older.

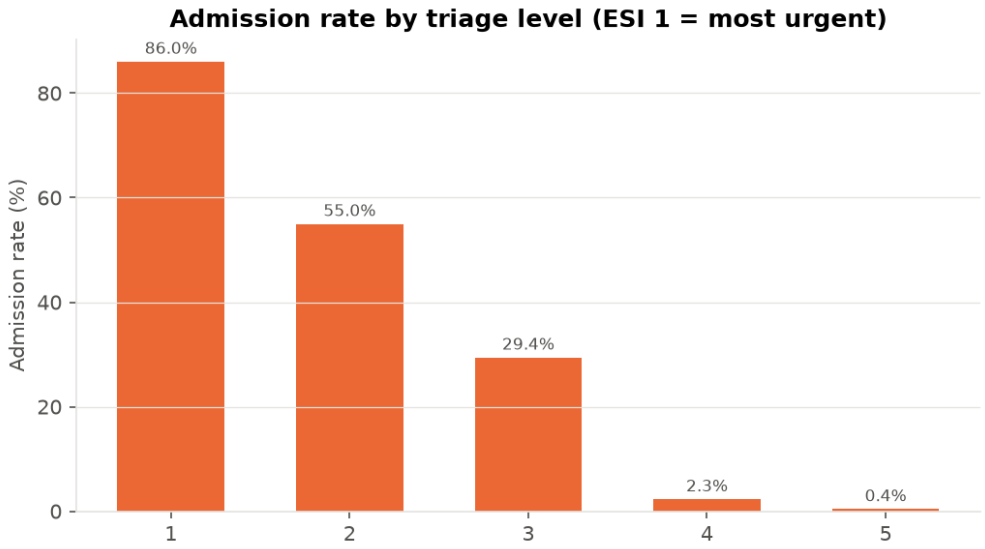


Figure 4: *Admission rate by triage level (ESI)*

The triage level is the strongest single signal (Figure 4): 86% of ESI-1 patients (resuscitation) were admitted, 55% of ESI-2, 29% of ESI-3 and less than 3% of ESI-4 and ESI-5 patients. However, ESI-3 is the largest group and its outcome is uncertain (about one in three admitted) - this is where the model adds most value.

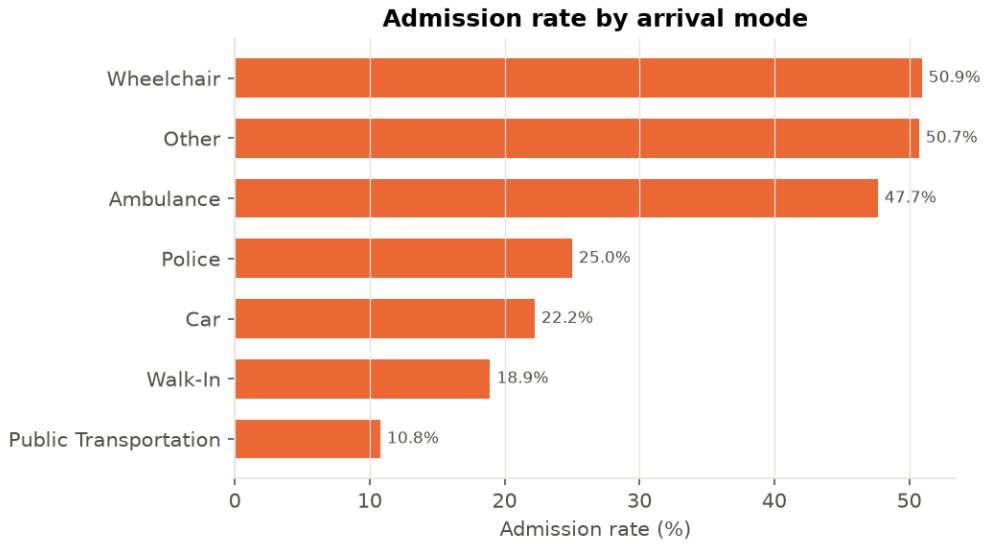


Figure 5: *Admission rate by arrival mode*

Patients arriving by ambulance were admitted in 47.7% of cases, compared with 18.9% for walk-in patients and 22.2% for those arriving by car (Figure 5). Arrival mode therefore reflects how sick the patient is.

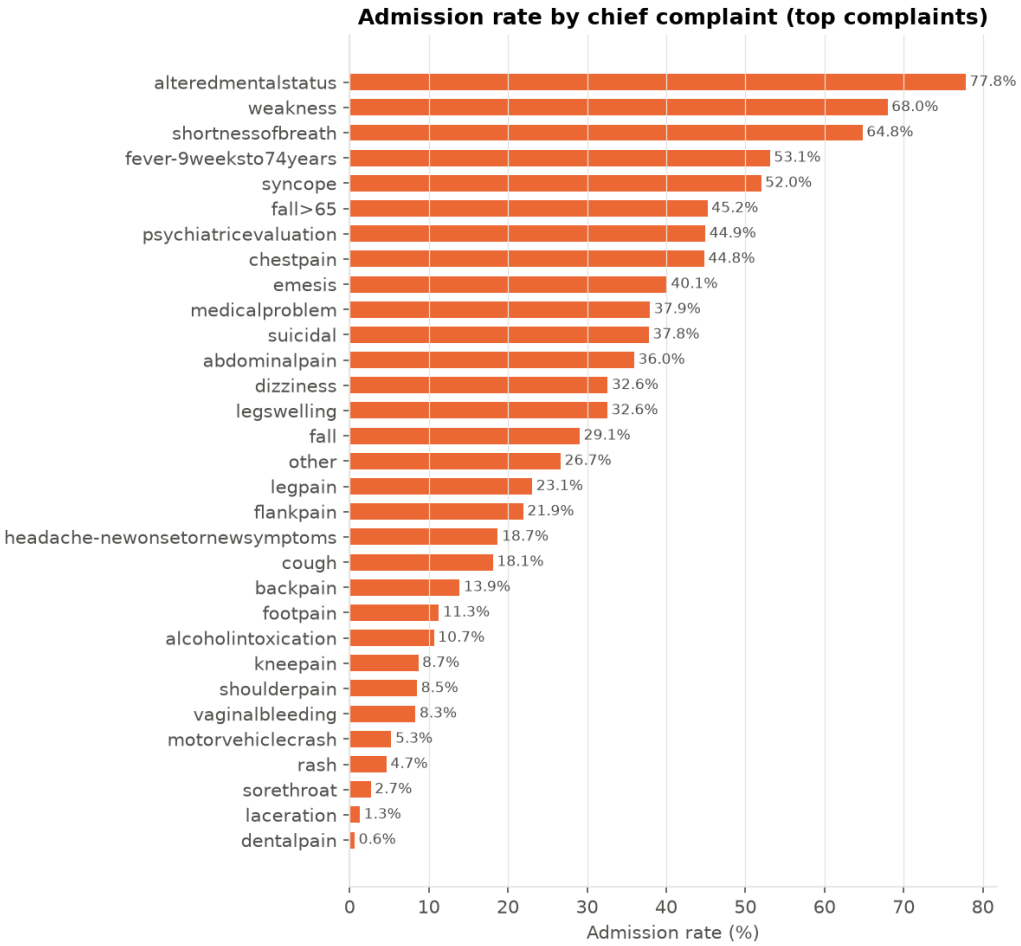


Figure 6: *Admission rate by chief complaint*

The chief complaint strongly changes the risk (Figure 6). Complaints such as alteredmentalstatus (78%), weakness (68%), shortnessofbreath (65%) lead to admission much more often than dentalpain (1%), laceration (1%), sorethroat (3%).

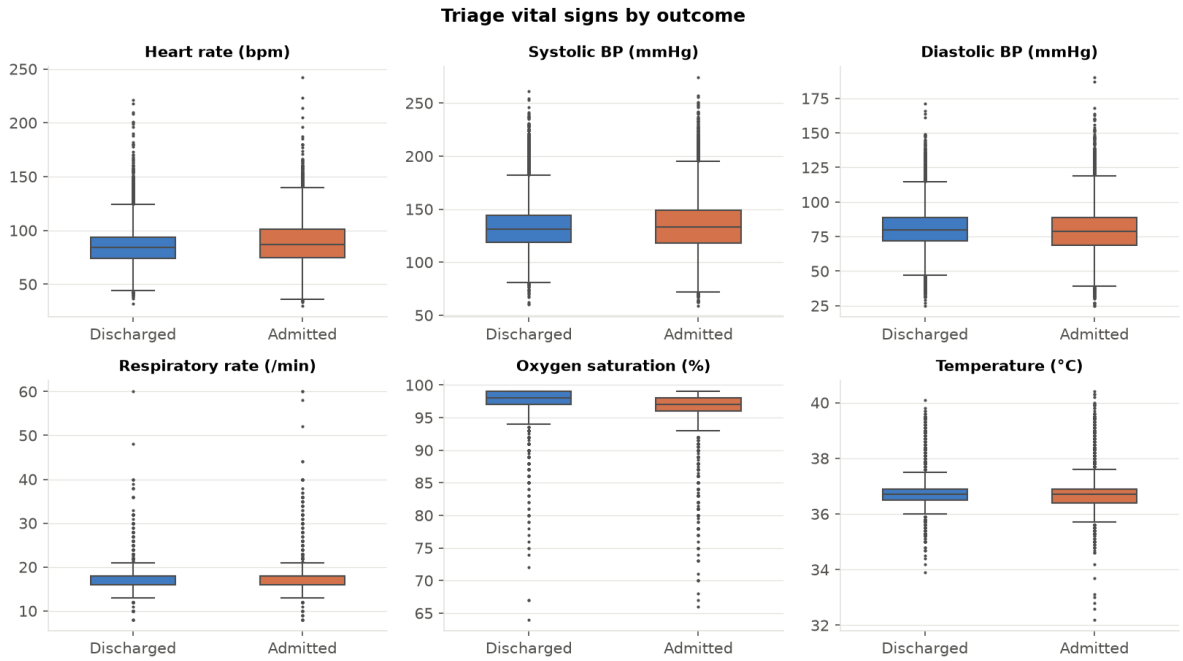


Figure 7: *Triage vital signs by outcome*

Figure 7 shows that admitted patients tend to have a higher heart rate and respiratory rate and lower oxygen saturation. The boxes overlap a lot, so a single vital sign cannot separate the two groups; the model must combine many weak signals.

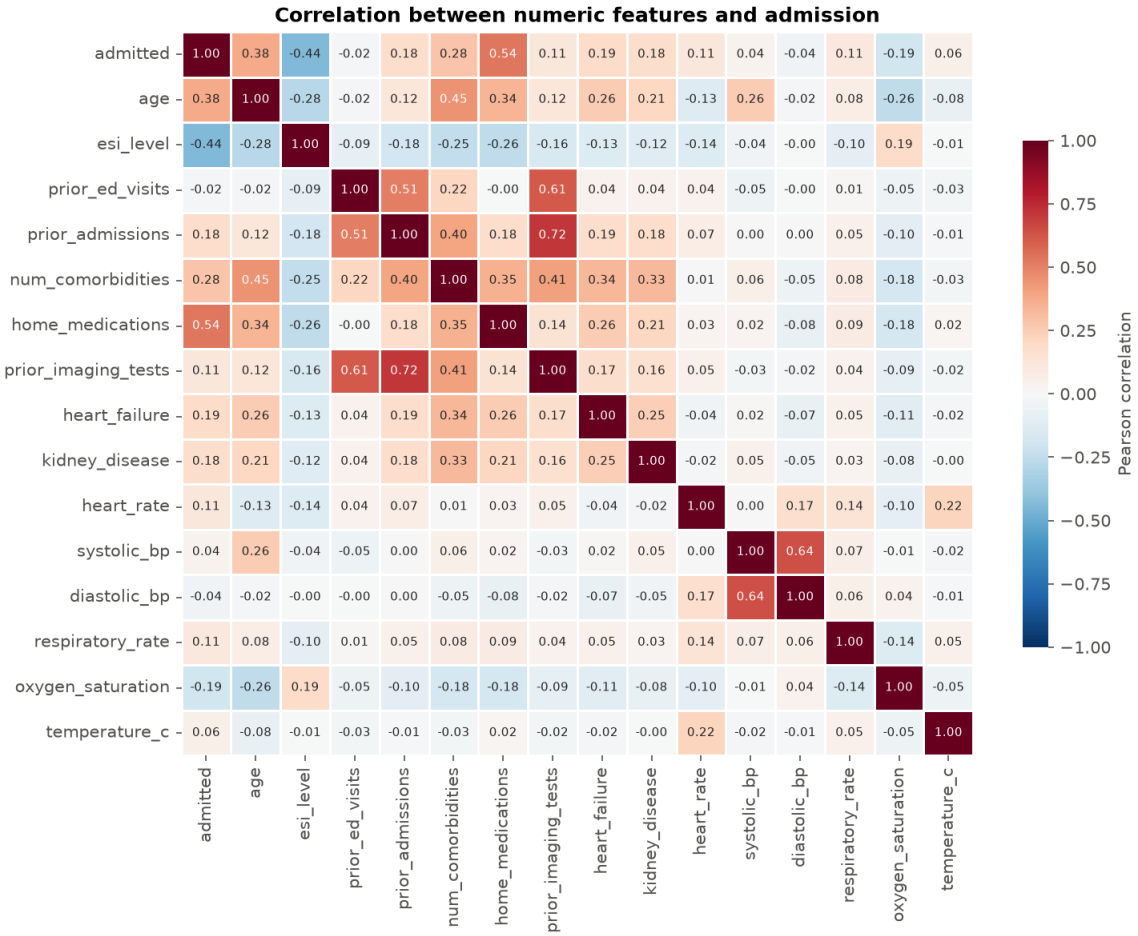


Figure 8: *Correlation heatmap of numeric features*

The correlation heatmap (Figure 8) confirms these findings: admission is negatively correlated with ESI level (r = -0.44, because ESI 1 is the most urgent) and positively with age (r = 0.38) and the number of comorbidities (r = 0.28). The home-medication count shows a suspiciously high correlation (r = 0.54), which was investigated as possible data leakage. History variables (prior ED visits, admissions, imaging) are correlated with each other, so tree-based models that cope well with correlated inputs are a good choice.

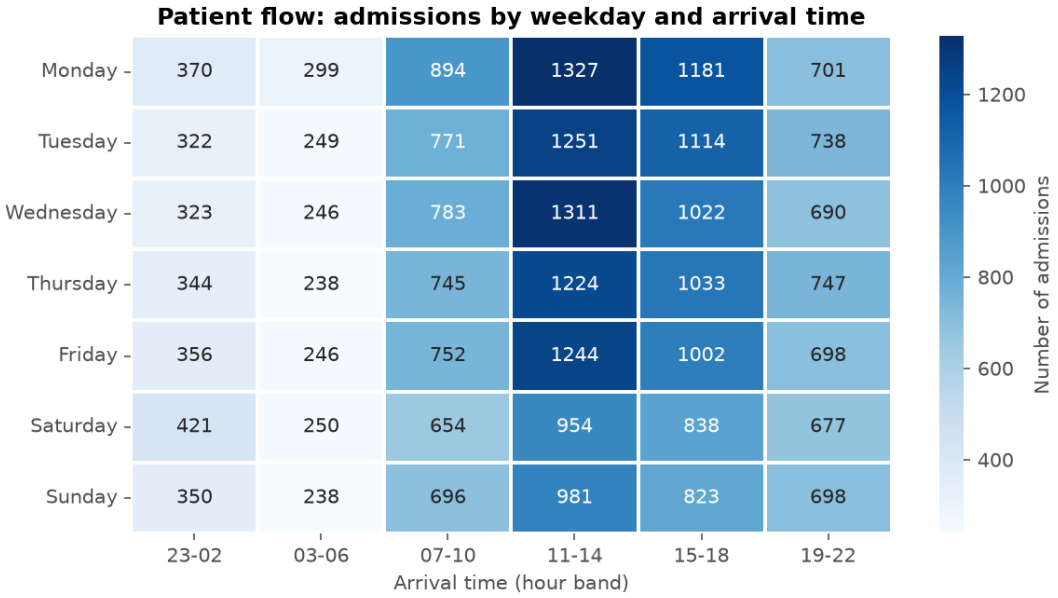


Figure 9: *Patient flow: number of admissions by weekday and arrival time*

Figure 9 is an operational view of patient flow. Most admissions come from patients who arrive between 11:00 and 18:59, and weekdays (especially Monday) are busier than weekends. Night arrivals (03:00-06:59) produce the fewest admissions. Bed managers can use this pattern to plan staff and discharges from wards before the afternoon peak.

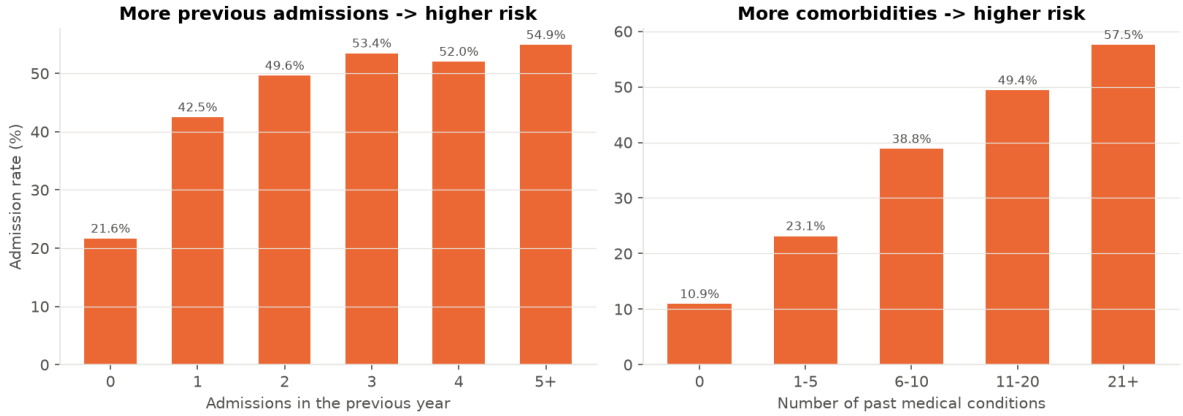


Figure 10: *Admission rate by previous admissions and number of past conditions*

Hospital history matters (Figure 10): patients with no admission in the previous year were admitted 21.6% of the time, but those with three or more previous admissions were admitted 53.9% of the time. The admission rate also grows with the number of chronic conditions.

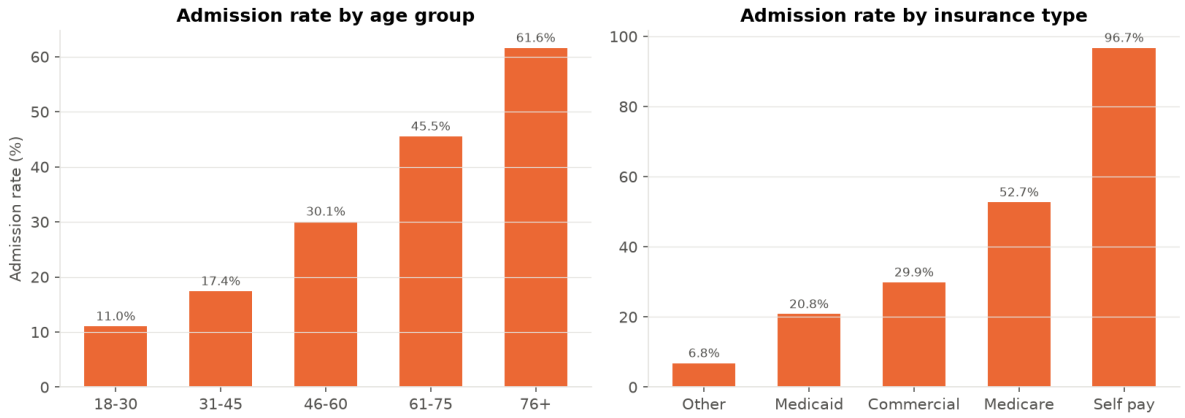


Figure 11: *Admission rate by age group and insurance type*

Figure 11 shows that Medicare patients (mostly elderly) have a high admission rate (52.7%). The very high rate of the 'Self pay' group (96.7%) is based on only 306 visits and probably reflects billing practice (insurance is registered after admission), so it is interpreted with caution.

7. Class Imbalance

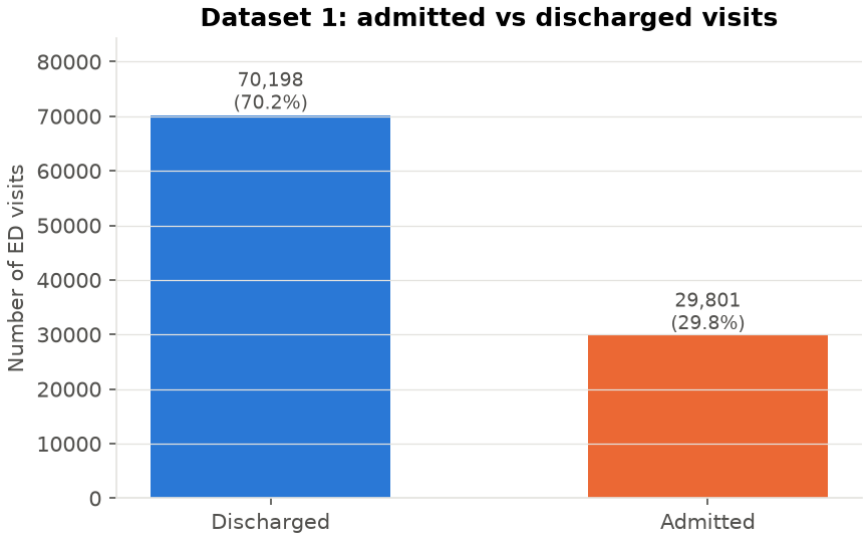


Figure 12: *Class distribution of Dataset 1*

Only 29.8% of visits end in admission, so there are 2.36 discharged patients for every admitted patient (Figure 12). This is a **moderate imbalance**. A useless model that always predicts 'discharge' would already be 70.2% accurate, so accuracy alone is misleading. The following techniques were used:

- **Stratified train/test split** (80/20) so both sets keep the same 29.8% admission rate.
- **class_weight = 'balanced'** in every model, so an error on an admitted patient costs about 2.4 times more than an error on a discharged patient during training.
- **Minority-focused metrics**: recall, precision, F1-score and PR-AUC in addition to accuracy.
- **Decision-threshold tuning** to choose the best trade-off between recall and precision.

The effect is clear for Logistic Regression: without balancing, recall for admitted patients was only 0.607; with balanced weights it rose to 0.805 and the F1-score improved from 0.659 to 0.696. Accuracy dropped slightly (0.812 to 0.790), which is acceptable because detecting admissions is the real goal. Oversampling methods such as SMOTE were not needed because the imbalance is moderate and the dataset is large.

8. Models Used

Training used 79,999 visits (80%) and testing used 20,000 unseen visits (20%). Four different classification algorithms were implemented, followed by two performance-improvement methods.

8.1 Logistic Regression

A linear model that estimates the log-odds of admission as a weighted sum of the features and converts it to a probability with the sigmoid function. It is fast, easy to interpret and a strong baseline. Settings: max_iter = 2000, class_weight = 'balanced'.

8.2 Decision Tree

A tree of if-then rules that splits patients into groups with similar outcomes using Gini impurity. To avoid over-fitting the depth was limited (max_depth = 10, min_samples_leaf = 50).

8.3 Random Forest

An ensemble of 200 decision trees, each trained on a bootstrap sample with a random subset of features; the final prediction is the average vote. Averaging reduces variance and over-fitting. Settings: n_estimators = 200, max_depth = 18, min_samples_leaf = 5, class_weight = 'balanced'.

8.4 Gradient Boosting (HistGradientBoostingClassifier)

Trees are added one at a time, each new tree correcting the errors of the previous ones. The histogram-based version in scikit-learn bins numeric values, which makes training on 80,000 rows fast and also supports missing values natively. It is usually the most accurate algorithm for tabular data.

8.5 Hyperparameter Tuning

The Gradient Boosting model was tuned with **RandomizedSearchCV**: 12 random combinations of hyperparameters were evaluated with 3-fold stratified cross-validation on the training set, using ROC-AUC as the score (195 seconds). Table 8 lists the search space and the best values found.

Table 8: *Hyperparameter search space and result*

Hyperparameter	Values searched	Best value
learning_rate	0.03, 0.05, 0.08, 0.1, 0.15	0.08
max_iter	200, 300, 400, 600	400
max_leaf_nodes	15, 31, 63, 127	31
min_samples_leaf	20, 50, 100, 200	100
l2_regularization	0, 0.1, 1, 5	5.0

Best mean cross-validation ROC-AUC: **0.8794** (standard deviation 0.0022), which shows that the result is stable across folds.

8.6 Ensemble (Soft Voting)

A soft-voting ensemble averages the predicted probabilities of Logistic Regression, Random Forest and the tuned Gradient Boosting model (weights 1 : 1 : 2). Combining different types of models can reduce individual errors.

Code 2: hyperparameter tuning with RandomizedSearchCV

```
search = RandomizedSearchCV(
    Pipeline([("prep", make_preprocessor()),
              ("model", HistGradientBoostingClassifier(class_weight="balanced",
                                                       random_state=RANDOM_STATE))]),
    param_distributions=param_dist, n_iter=12, scoring="roc_auc",
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1, random_state=RANDOM_STATE, verbose=0)
```

9. Results and Evaluation

9.1 Model Comparison

Table 9: *Performance of all models on the test set (20,000 visits, threshold 0.5)*

Model	Accuracy	Precision	Recall	Specificity	F1	ROC-AUC	PR-AUC
Logistic Regression (no balancing)	0.812	0.719	0.607	0.899	0.659	0.877	0.744
Logistic Regression	0.790	0.613	0.805	0.784	0.696	0.877	0.743
Decision Tree	0.751	0.556	0.813	0.724	0.660	0.855	0.693
Random Forest	0.789	0.612	0.792	0.787	0.691	0.875	0.728
Gradient Boosting	0.790	0.608	0.826	0.774	0.701	0.884	0.757
Tuned Gradient Boosting	**0.792**	**0.611**	**0.829**	**0.776**	**0.703**	**0.885**	**0.758**
Voting Ensemble	0.794	0.616	0.821	0.782	0.703	0.885	0.753

All balanced models reach a recall of about 0.80 and a ROC-AUC between 0.855 and 0.885. The single Decision Tree is the weakest. **Tuned Gradient Boosting** achieved the best ROC-AUC (0.885) and was selected as the final model. Tuning improved the ROC-AUC of Gradient Boosting from 0.8845 to 0.8853; the voting ensemble (0.8848) performed almost identically, so the simpler single model was preferred. The final model meets both success criteria (ROC-AUC > 0.85 and recall > 0.80).

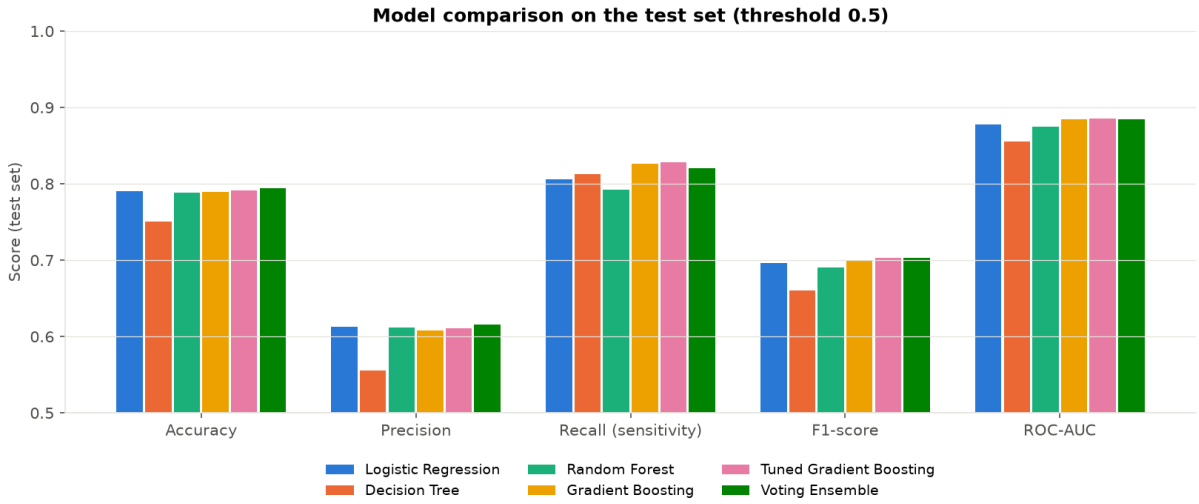


Figure 13: *Comparison of the main models on the test set*

9.2 ROC and Precision-Recall Curves

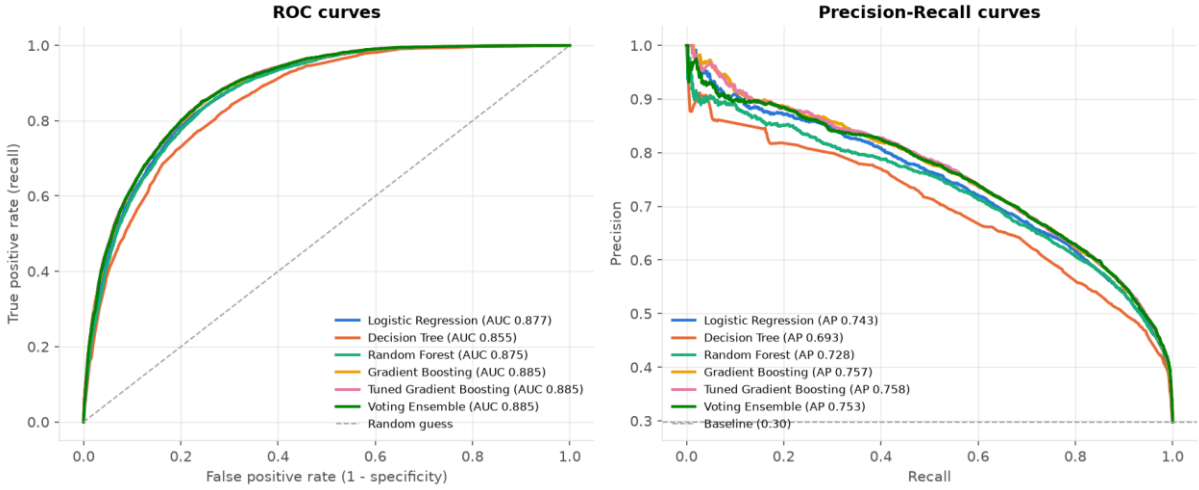


Figure 14: *ROC curves (left) and Precision-Recall curves (right)*

The ROC curve (Figure 14, left) shows the trade-off between catching admissions (true positive rate) and false alarms (false positive rate) for every threshold. All curves are far above the diagonal (random guessing, AUC 0.5). The Precision-Recall curve (right) is more informative for imbalanced data; the best model's PR-AUC of 0.758 is much higher than the baseline of 0.30 (the admission rate).

9.3 Confusion Matrices

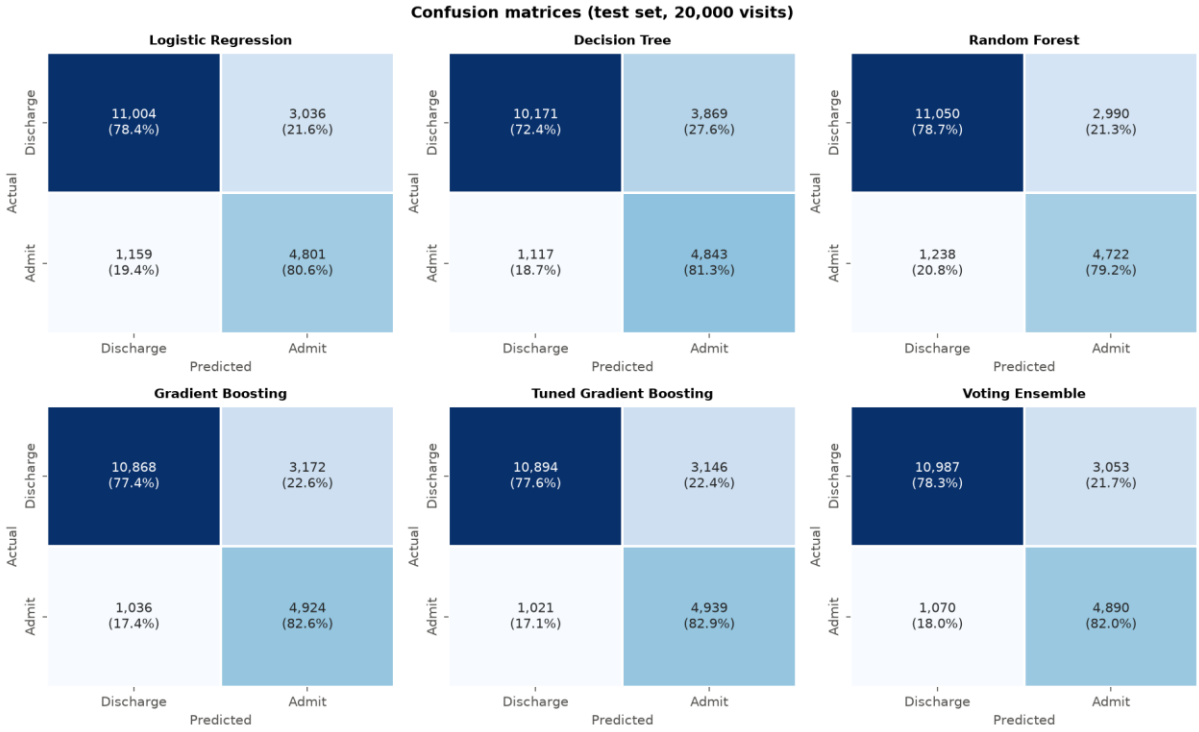


Figure 15: *Confusion matrices of all main models (test set)*

For the final model, about **4,939 of 5,960 admitted patients** were correctly identified (recall 82.9%) and about **10,894 of 14,040 discharged patients** were correctly identified (specificity 77.6%). The remaining errors are mostly borderline cases (ESI 3 patients with moderate vital signs) where even clinicians need test results before deciding.

9.4 Decision-Threshold Tuning

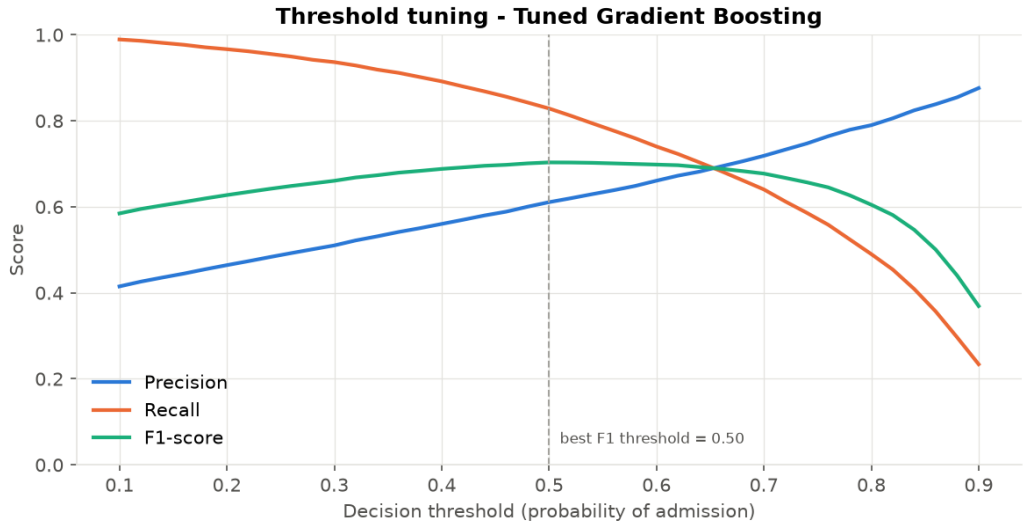


Figure 16: *Precision, recall and F1-score for different decision thresholds*

The default threshold of 0.5 is not always the best choice. Figure 16 shows that the F1-score is highest at a threshold of **0.50**. A hospital can move the threshold depending on its goal: at 0.40 the model catches 89% of admissions (precision 56%), useful for early bed requests; at 0.70 precision rises to 72% but recall falls to 64%. The GUI uses three risk bands (LOW < 0.40, MODERATE 0.40-0.50, HIGH ≥ 0.50) based on this analysis.

9.5 Feature Importance

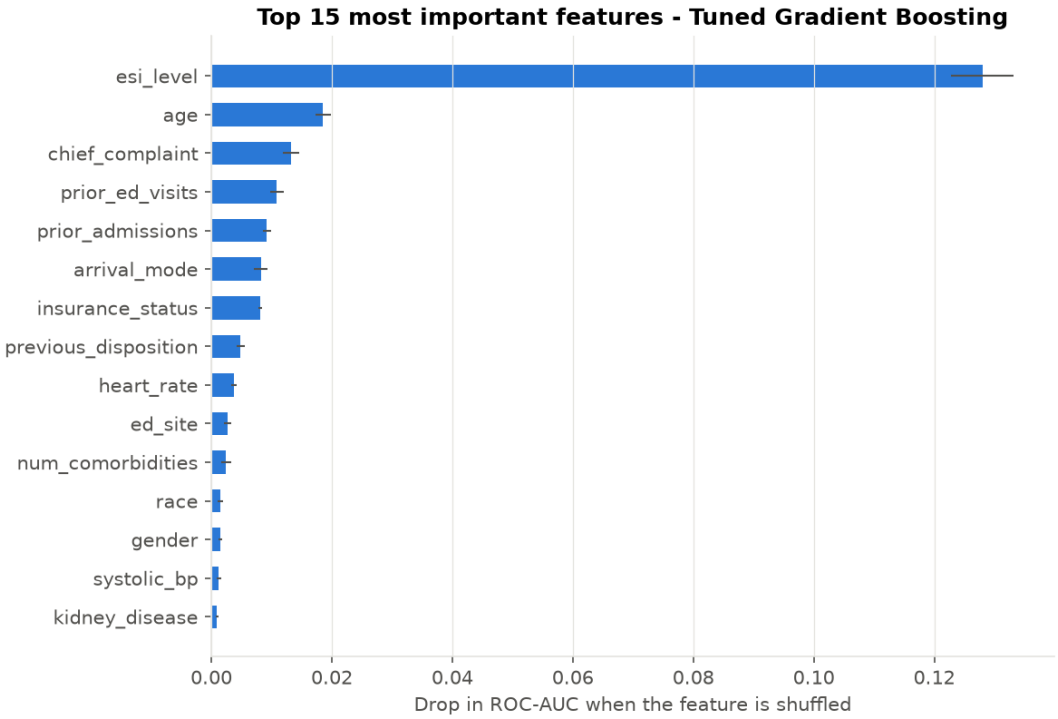


Figure 17: *Permutation feature importance of the final model*

Permutation importance measures how much the test ROC-AUC drops when the values of one feature are randomly shuffled. The most important features are **esi_level** (0.128), **age** (0.019), **chief_complaint** (0.013), **prior_ed_visits** (0.011), **prior_admissions** (0.009). This agrees with clinical intuition and with the EDA: urgency, age, the main complaint and past hospital use drive admission decisions.

9.6 Data-Leakage Check

During EDA the number of home medications showed an unusually strong relationship with admission (admitted patients: 6.1 medications on average, discharged: 0.5). In hospitals the medication list is completed during **medication reconciliation**, which happens as part of the admission process. The feature therefore partly records the outcome itself and would not be reliably available at triage.

Table 10: *Effect of the leaky feature on model performance*

Feature set (tuned Gradient Boosting)	Test ROC-AUC
With home_medications (leaky)	0.9271
Final model (without)	0.8853

Adding the feature increases ROC-AUC from 0.885 to 0.927, an over-optimistic result that would not hold in real use. The feature was therefore **excluded**, and all results in this report are for the honest model. The final performance is in line with the original study by Hong et al. (2018), which reported ROC-AUC values between about 0.87 and 0.92 depending on which variables were used.

10. Dataset 2 and Cross-Dataset Validation

10.1 Korean KTAS Dataset

Dataset 2 was cleaned with the same principles: the Korean text '측불' (unmeasurable) found in the vital-sign columns (for example 25 systolic BP and 20 heart-rate values) was converted to missing values, the pain score was set to 0 for patients without pain, and transfers/deaths were removed. Figure 18 shows the class distribution (32.9% admitted), the admission rate by nurse KTAS level (from 91% for KTAS 1 to 8% for KTAS 5) and the age difference between the groups.

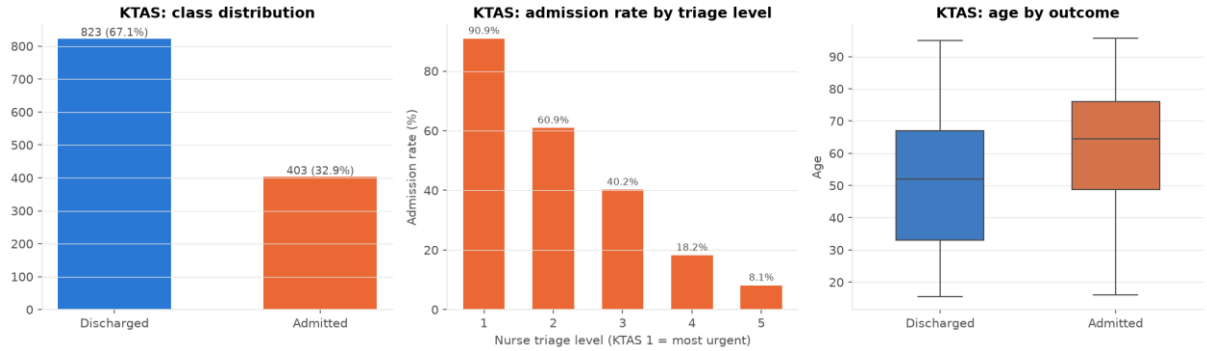


Figure 18: *EDA of the Korean KTAS dataset*

Because the dataset is small, models were evaluated with **5-fold stratified cross-validation** instead of a single split, which gives a more reliable estimate.

Table 11: *KTAS models - 5-fold stratified cross-validation*

Model	Accuracy	Precision	Recall	F1	ROC-AUC	AUC std
Nurse KTAS level only (baseline)	0.644	0.473	0.759	0.583	0.711	± 0.045
Logistic Regression	0.737	0.584	0.715	0.642	0.799	± 0.045
Random Forest	**0.745**	**0.601**	**0.682**	**0.638**	**0.808**	**± 0.028**
Gradient Boosting	0.746	0.605	0.682	0.639	0.794	± 0.030

Using only the nurse's KTAS level gives a ROC-AUC of 0.711. Adding age, vital signs, arrival mode, mental state and pain information raises it to 0.808 (Random Forest), and accuracy from 0.644 to 0.745. This proves that machine learning adds value over the triage scale alone, also in a second country.

10.2 External Validation (USA → Korea)

To test generalisation, a model was trained on the USA data using only the ten triage features that both datasets share (age, sex, triage level, ambulance arrival, heart rate, systolic and diastolic BP, respiratory rate, temperature in °C and oxygen saturation). ESI and KTAS are both five-level scales where 1 is the most urgent, so they were mapped directly. The model was then tested on the Korean patients, which it had never seen.

Table 12: *External validation of the triage-only model*

Test data	ROC-AUC
USA test set (same hospital system)	0.848
Korea KTAS (external, never seen)	0.760

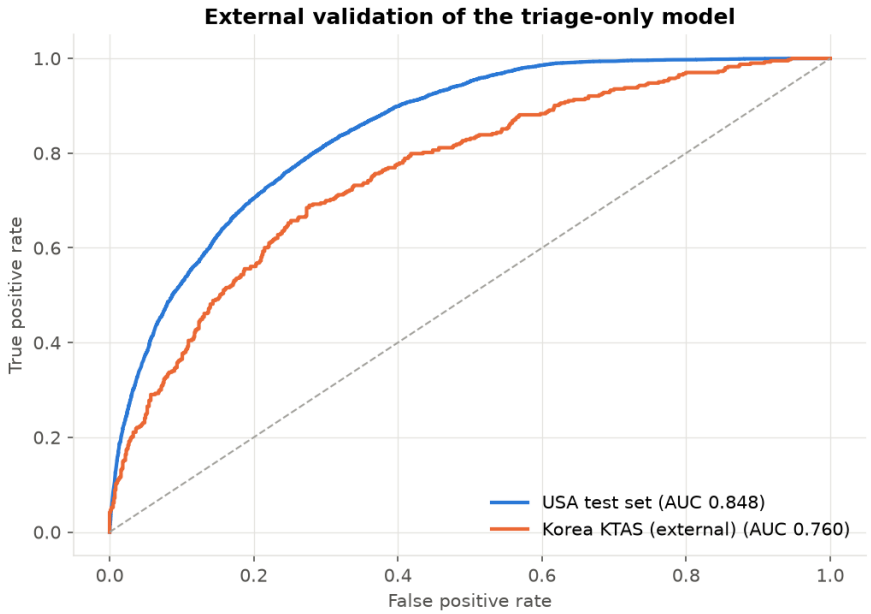


Figure 19: *ROC curves of the triage-only model on USA and Korean data*

The model keeps useful discrimination on Korean patients (ROC-AUC 0.760) although it was trained in another country with another triage scale, but performance drops from 0.848. Differences in patient populations, triage practice and admission policies explain the gap. In practice, a model should therefore be fine-tuned and re-calibrated on local data before deployment.

11. Visualization and User Interface

The brief asks for a simple, user-friendly visualization or interface. The project provides three levels of visualization: (1) 19 charts saved automatically in outputs/figures, (2) a desktop application (03_admission_app.py) for ED staff and bed managers, and (3) a bed-demand forecast.

11.1 Admission Prediction App

The app was built with **Tkinter**, which is included with Python, so no extra installation is needed. The 'Predict Admission' tab contains a form grouped like a triage sheet (patient, arrival & triage, vital signs, history, known conditions). After pressing **Predict**, the app shows the risk level (LOW / MODERATE / HIGH) in a coloured box, the decision, the model score as a progress bar, a calibrated probability and a recommended action. The button **Load random test patient** fills the form with a real visit from the unseen test set and shows its true outcome, which is useful for demonstrations. Invalid input (for example letters in the age field) is caught and explained.

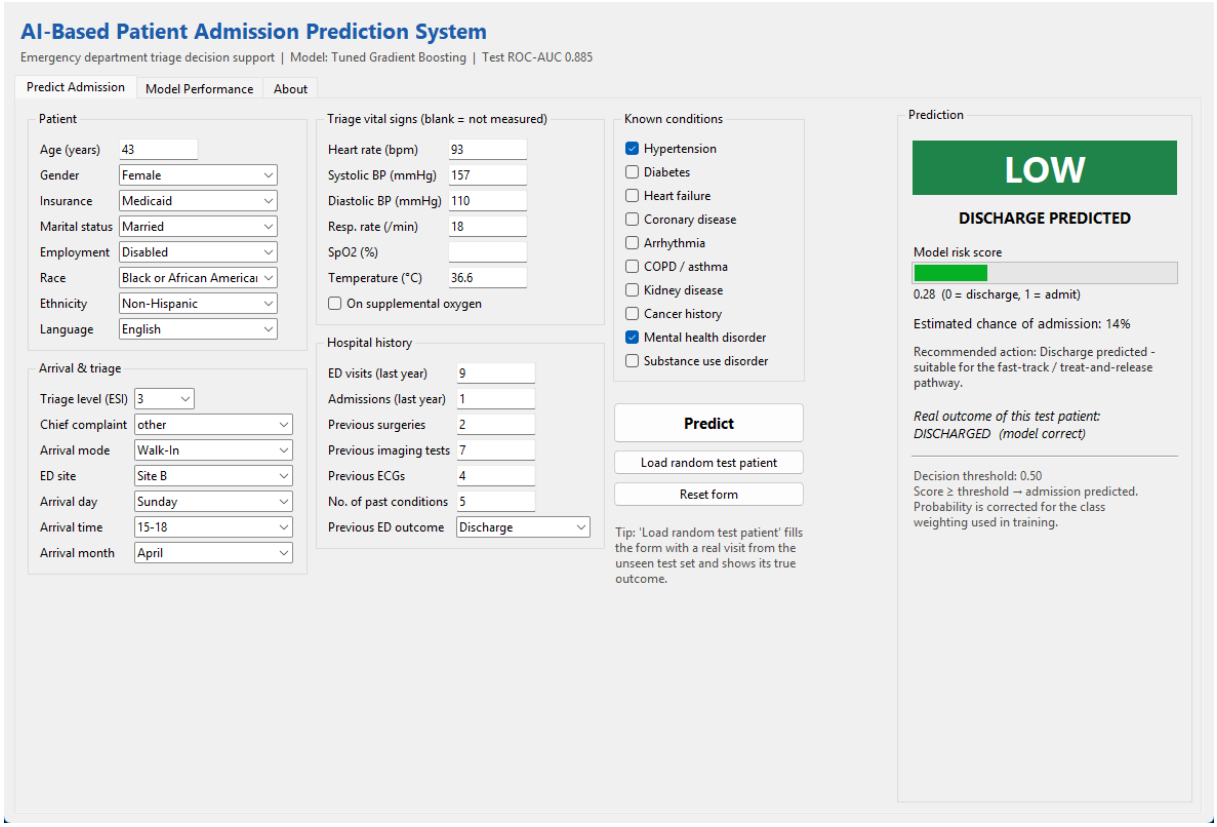


Figure 20: *Prediction tab of the Admission Prediction App*

The 'Model Performance' tab shows the comparison tables for both datasets and a chart viewer for all project figures, so users can understand how reliable the model is. The 'About' tab lists the datasets, the group members and a clinical disclaimer.

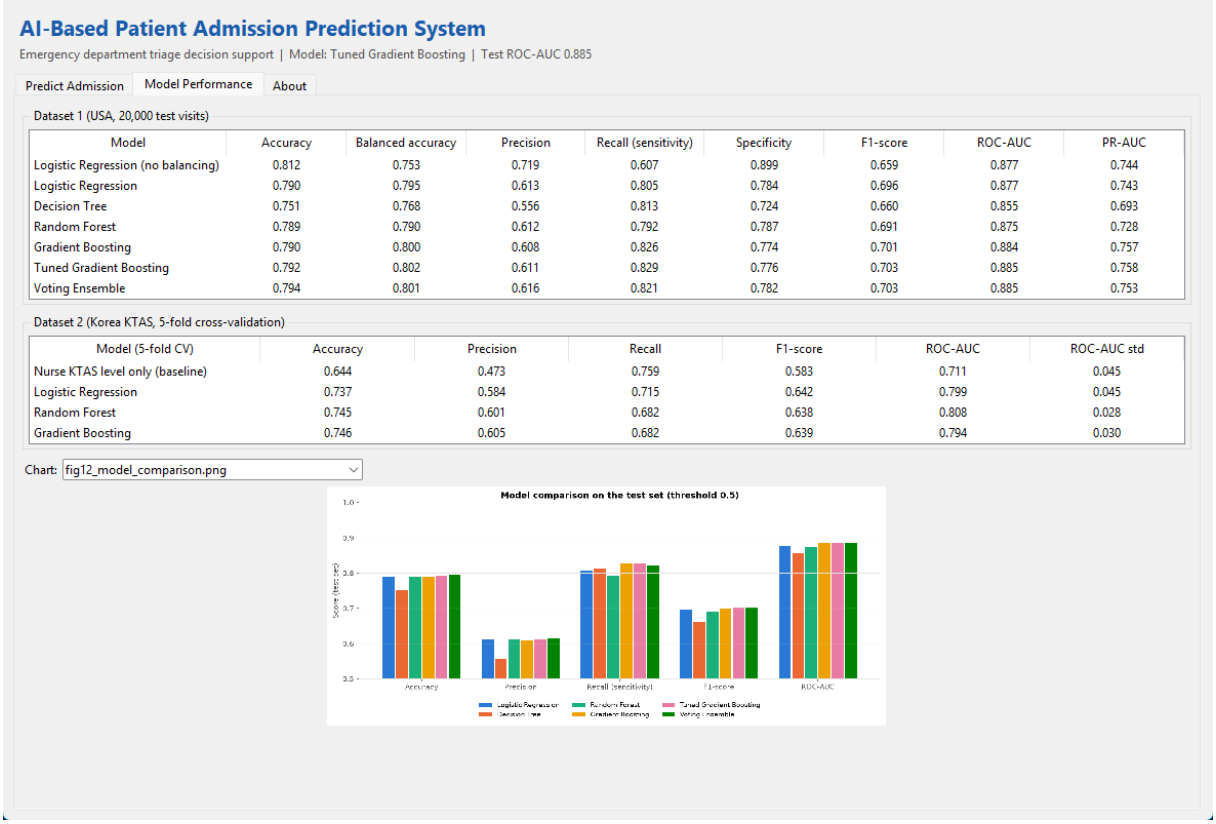


Figure 21: *Model Performance tab with tables and chart viewer*

11.2 Bed-Demand Forecast

Because the model is trained with balanced class weights, its raw scores are shifted upwards (the sum of raw scores over the test set predicts 8,134 admissions while only 5,960 happened). The scores were therefore **calibrated** by dividing the predicted odds by the class ratio (2.36). Summing the calibrated probabilities gives a forecast of **5,891 admissions** - within 1.2% of the real number.

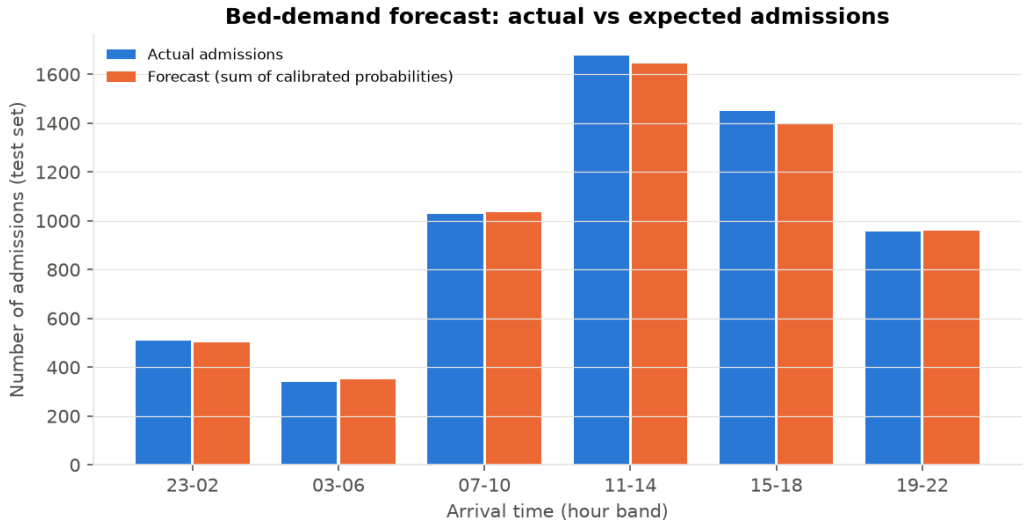


Figure 22: *Actual vs forecast admissions per arrival time band*

Table 13: *Bed-demand forecast by arrival time band (test set)*

Arrival time	Actual admissions	Forecast	Error
23-02	509	501	-1.6%
03-06	340	351	+3.2%
07-10	1028	1036	+0.8%
11-14	1678	1643	-2.1%
15-18	1449	1401	-3.3%
19-22	956	959	+0.3%

A bed manager can add up the calibrated probabilities of all patients currently waiting in the ED to estimate how many beds will be needed in the next hours - a direct, actionable use of the model for better hospital operations.

12. Challenges and Solutions

Table 14: *Main challenges and how they were solved*

Challenge	Solution
Very large raw file (972 columns, 560k rows, 7.4 GB in memory)	Summarised related columns and sampled 100,000 visits; preparation done once in a separate script.
Data stored in R format (.RData)	Read in Python with the pyreadr package and saved as CSV.
Many missing vital signs (30-48%)	Median imputation with missing-indicator columns instead of deleting rows.
Text in numeric columns (Korean '측불')	Converted with pd.to_numeric(errors='coerce') after inspecting the values.
Inconsistent categories and units	Standardised spelling, converted °F to °C, unified placeholders.
Class imbalance	Stratified split, class weights, PR-AUC/F1, threshold tuning.
Data leakage (home medications)	Detected in EDA, confirmed with an experiment, excluded from the model.
Over-confident probabilities after class weighting	Prior-correction calibration for the bed-demand forecast.
Long training time for tuning	RandomizedSearchCV (12 settings) instead of a full grid; fast histogram boosting; n_jobs = -1.
Different triage scales and units in two countries	Built a harmonised set of 10 shared features for external validation.

13. Ethical Considerations and Limitations

- **Privacy:** both datasets are de-identified and publicly released for research; no names, dates of birth or record numbers are present. The project runs locally and no data is sent to external services.
- **Fairness:** demographic variables (race, insurance) were included because they were in the source data, but they have low importance. A real system should audit performance per group so that no group is disadvantaged.

- **Human in the loop:** the system only supports decisions. The final admission decision remains with clinicians, and the GUI shows a clear disclaimer.
- **Limitations:** data from 2014-2017 and from specific hospitals; admission policies differ between hospitals and change over time; the external validation showed a drop in performance; lab results and free-text notes were not used.

14. Conclusion and Future Work

This project developed a complete AI-based patient admission prediction system in Python. Two real-world datasets were cleaned and analysed, the main factors behind admission were identified (triage level, age, chief complaint, hospital history, arrival mode and vital signs), and several machine-learning models were trained, tuned and compared. The final Tuned Gradient Boosting model predicts admission at triage with a ROC-AUC of 0.885, recall of 82.9% and accuracy of 79.2% on 20,000 unseen visits, after removing a leaky feature that would have inflated the results. Validation on a Korean dataset showed that the approach generalises, with some loss in accuracy. The Tkinter application and the calibrated bed-demand forecast turn the model output into information that ED staff and bed managers can act on.

Future work:

- Add early laboratory results and free-text triage notes (natural language processing).
- Re-train and calibrate the model on data from a local (for example Karachi) hospital.
- Explain individual predictions with SHAP values in the interface.
- Integrate the model with the hospital information system for real-time bed dashboards.
- Monitor fairness and model drift over time.

References

- Hong WS, Haimovich AD, Taylor RA. Predicting hospital admission at emergency department triage using machine learning. PLOS ONE. 2018;13(7):e0201016. Data: https://github.com/yaleemmlc/admissionprediction (Zenodo record 1308993).
- Moon SH, Shim JL, Park KS, Park CS. Triage accuracy and causes of mistriage using the Korean Triage and Acuity Scale. PLOS ONE. 2019;14(9):e0216972. Supporting information S1 Data.
- Pedregosa F, et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research. 2011;12:2825-2830.
- McKinney W. Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference. 2010.
- Hunter JD. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering. 2007;9(3):90-95.
- Waskom ML. seaborn: statistical data visualization. Journal of Open Source Software. 2021;6(60):3021.
- Chen T, Guestrin C. XGBoost: A Scalable Tree Boosting System. Proceedings of KDD. 2016 (gradient boosting background).

Appendix A: Project Files and How to Run

Table 15: *Project files*

File / folder	Purpose
01_prepare_datasets.py	Builds the two CSV files from the raw downloads (run once; CSVs already included)
02_admission_prediction.py	Main project: preprocessing, EDA, imbalance, models, tuning, evaluation, validation
03_admission_app.py	Tkinter user interface for predictions and results
admission_utils.py	Shared cleaning functions and constants used by the scripts and the app
data/ed_patient_flow.csv	Dataset 1 - 100,000 ED visits (USA)
data/ktas_triage.csv	Dataset 2 - 1,267 ED visits (Korea)
outputs/figures/	All 19 charts produced by the main script
outputs/metrics/	Result tables (CSV): model comparison, tuning, feature importance, etc.
outputs/models/	Saved final model, model information and demo patients for the app
requirements.txt / README.md	Library list and instructions

Steps to run:

```
pip install -r requirements.txt
python 02_admission_prediction.py      # about 5.4 minutes, creates outputs/
python 03_admission_app.py             # opens the prediction interface
```

Appendix B: Key Code

Code 3: cleaning of the Korean KTAS dataset (admission_utils.py)

```
def clean_ktas(df):
    """Clean Dataset 2 (Korean KTAS emergency department records)."""
    df = df.copy()
    df.columns = [c.strip() for c in df.columns]


    # Vital signs contain the Korean word '측불' (= "unmeasurable") -> NaN
    for col in ["SBP", "DBP", "HR", "RR", "BT", "Saturation"]:
        df[col] = pd.to_numeric(df[col], errors="coerce")


    limits = {"SBP": (50, 300), "DBP": (20, 200), "HR": (20, 250), "RR": (5, 60),
              "BT": (32, 43), "Saturation": (50, 100)}
    for col, (low, high) in limits.items():
        df.loc[(df[col] < low) | (df[col] > high), col] = np.nan


    # Pain score is only recorded when the patient has pain; no pain -> score 0
    df.loc[df["Pain"] == 2, "NRS_pain"] = df.loc[df["Pain"] == 2, "NRS_pain"].fillna(0)


    # Transfers and deaths are neither 'admitted here' nor 'sent home' -> removed
    df = df[df["Disposition"].isin(KTAS_ADMIT_CODES + KTAS_NOT_ADMIT_CODES)].copy()
    df[TARGET] = df["Disposition"].isin(KTAS_ADMIT_CODES).astype(int)
    return df
```

Code 4: evaluation function with all metrics

```
def evaluate(y_true, prob, threshold=0.5):
    pred = (prob >= threshold).astype(int)
```

```
    tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
    return {
        "Accuracy": accuracy_score(y_true, pred),
        "Balanced accuracy": balanced_accuracy_score(y_true, pred),
        "Precision": precision_score(y_true, pred),
        "Recall (sensitivity)": recall_score(y_true, pred),
        "Specificity": tn / (tn + fp),
        "F1-score": f1_score(y_true, pred),
        "ROC-AUC": roc_auc_score(y_true, prob),
        "PR-AUC": average_precision_score(y_true, prob),
    }
```

Code 5: prediction of one patient in the app

```
    def predict(self, patient):
        row = pd.DataFrame([{c: patient.get(c, np.nan) for c in FEATURES}])
        row, _ = clean_patient_flow(row, self.info["top_complaints"])
        score = float(self.model.predict_proba(row[FEATURES])[:, 1][0])
        probability = float(calibrate_probability(score, self.info["prior_ratio"]))
        band, action = risk_band(score, self.threshold)
        return {"score": score, "probability": probability, "band": band, "action": action,
                "admit": score >= self.threshold}
```


THANK
YOU